# FINAL PROJECT REPORT

SEMESTER 1, ACADEMIC YEAR: 2024-2025

CT313H: WEB TECHNOLOGIES AND SERVICES

- **Project/Application name: Restaurant Ordering & Reservation Website**
- **GitHub links (for both frontend and backend): https://github.com/24-25Sem1-Courses/ct313h03-project-ngothuythanhtam.git**
- **Video Demo link: https://youtu.be/qkObzXKYE5U**
- **Student ID 1: B2111949**
- **Student Name 1: Ngô Thụy Thanh Tâm**
- **Student ID 2: B2111959**
- **Student Name 2: Nguyễn Thị Hoài Thương**
- **Class/Group Number: CT313HM03**

**CONTENTS**


I. Introduction ..... 2

1. Project/application description: ..... 2
2. A list of tables and their structures in the database ..... 2
3. A task assignment sheet for each member. ..... 3

II. Details of implemented features ..... 4

1. Feature / Application page 1: Menu Mangement ..... 4
2. Feature / Application page 2: Table ..... 7
3. Feature / Application page 3: Reservation ..... 9
4. Feature / Application page 4: Receipt ..... 14
5. Feature / Application page 5: User ..... 21

# I. Introduction

## 1. Project/application description:

The Restaurant Ordering & Reservation Website is an user – friendly platform desinged to streamline both table reservations and food orders for customers. It enables customers to place orders online, browse the menu, and either enjoy their meal at the restaurant or arrange for self-pickup.

## 2. A list of tables and their structures in the database

Our project has 6 tables, which are:

1. users
2. menu_items
3. restaurant_table
4. reservation
5. receipt
6. order_item

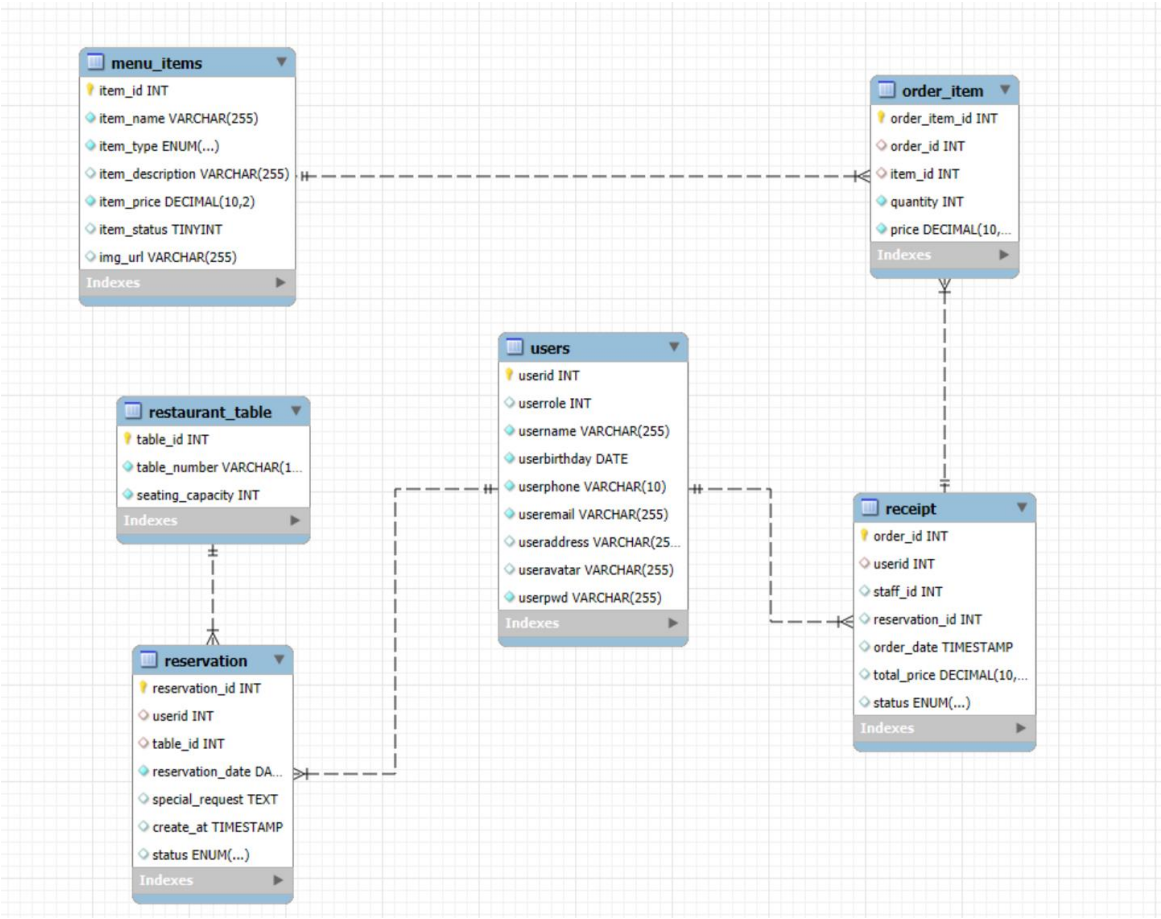


*Image 1. Entity-Relationship Diagram*

3. **A task assignment sheet for each member.**

| Name | Assigned task |
|---|---|
| **TamB2111949** | - Set up project repository.<br>- Set up database and design table structures.<br>- Write database queries for fetching data.<br>- Ensure data consistency and optimize queries for performance.<br>- Develop service and controller functions for menu items (CRUD), users (GET), reservations (CREATE, UPDATE, GET) and tables (CREATE, GET, DELETE), receipts (GET, CREATE, UPDATE).<br>- Define routes and document API endpoints with Swagger, including parameters, responses, and error handling.<br>- Test CRUD operations for menu items, reservations, tables, receipts and GET users.<br>- Write detailed project documentation.<br>- In charge of designing the UI and implementing the logic service for staff interface, including features for managing the menu, tables, reservations, and receipts. |
| **ThuongB2111959** | - Set up project repository<br>- Set up database and design table structures<br>- Write database queries for fetching data<br>- Ensure data consistency and optimize queries for performance<br>- Develop service and controller functions for menu items (GET), users (CRUD), reservations (POST, UPDATE, GET), receipts (CRUD) and tables (GET).<br>- Define routes and write document API endpoints, including parameters and responses (Swagger), APIs error.<br>- Test CRUD operations for users, receipts. |

| | |
|---|---|
| | - Test POST, GET, UPDATE for reservations.<br>- Test GET for menu items and tables.<br>- In charge of designing the UI and implementing the logic service for customer interface.<br>- Write detailed project documentation. |

## II. Details of implemented features

### 1. Feature / Application page 1: Menu Mangement

**Description:**

– In staff role: The menu management feature provides staff with tools to view and maintain the restaurant's menu. This includes viewing, adding new items, editing existing ones, and removing items. Validation checks minimize errors, ensuring data consistency.

– In customer role: The menu management feature provides customer with tools to view, search the restaurant's menu and add item from menu to Cart.

**Screenshots:**

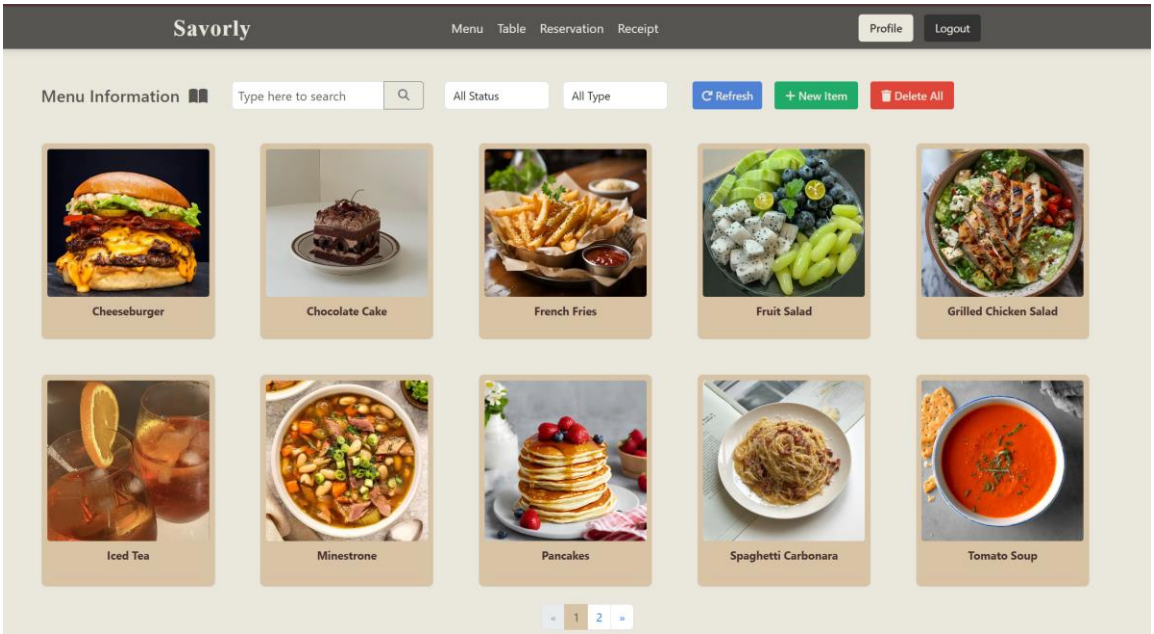


*Image 2. Menu before deleting Cheeseburger*

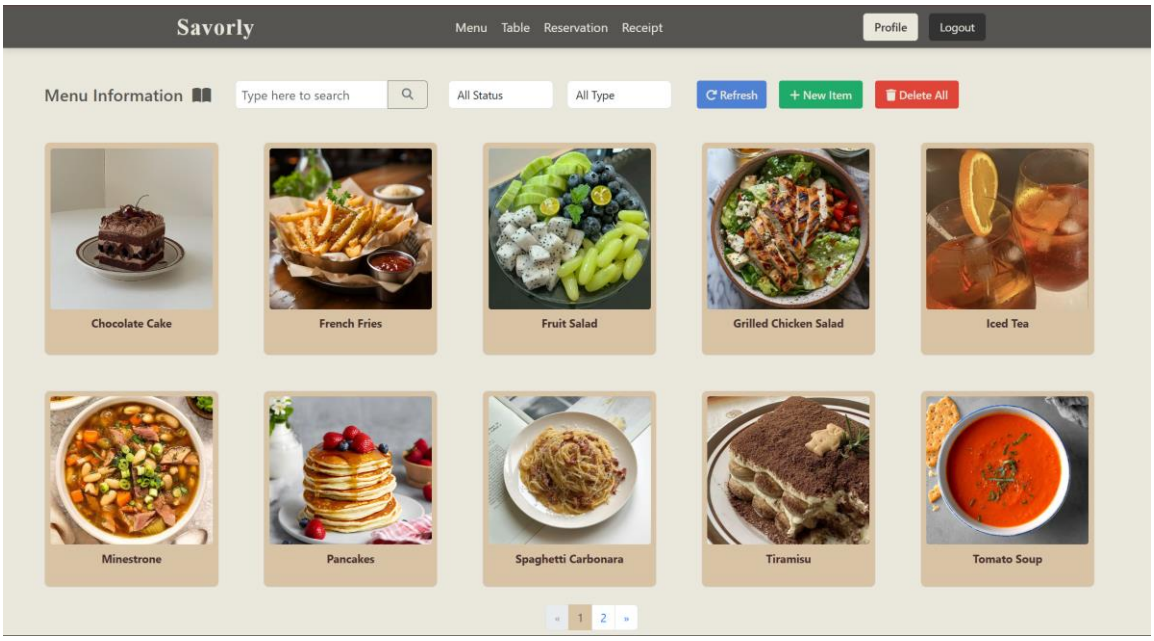


*Image 3. Menu after deleting Cheeseburger*

**Implementation details:**

<u>Libraries Used:</u>

- **Express**: A lightweight web framework used for creating API routes. It's responsible for handling HTTP requests and responses for the CRUD operations.
- **Knex**: A SQL query builder that simplifies database interactions, allowing me to perform SQL operations.
- **ApiError**: To handle errors.
- **fontawesome:** Used to add icons for actions like edit, delete, and view in the menu item table, enhancing the UI.
- **tanstack/vue-query:** Manages server-state for menu items by handling data fetching, caching, and synchronization, ensuring the menu list stays up-to-date without excessive API calls.
- **vee-validate/zod:** Validates the form for adding or editing menu items by enforcing rules like required fields, proper price format, and valid descriptions.

- **Bootstrap**: Provides styling for the menu item table, forms, and buttons, ensuring a professional layout.
- **SweetAlert2**: Displays confirmation dialogs when deleting menu items, as well as success or error alerts for CRUD operations.
- **Vue**: The core framework used to create components for displaying and managing menu items (ItemcCard, ItemForm, ItemList,…).
- **Vue-Router**: Handles navigation between menu views and its components.

**Server-Side APIs**:

DELETE /api/menu_items/ByStaff/{item_id}

**Description:** Staff delete item by id.

**Endpoint**: /api/menu_items/ByStaff/{item_id}

**Parameters (required):** item_id (integer)

**Format Received:**

```
{
  "status": "success",
  "data": {
    "table": {
      "table_id": 0,
      "table_number": "string",
      "seating_capacity": 0
    }
  }
}
```

**Data Storage:**

This feature reads and stores data in the menu_items table. The table fields include item_id, item_name, item_type, item_description, item_price, item_status, and img_url. All database operations are managed using Knex. Image files are uploaded via Multer and stored on the server. When a menu item is deleted, the associated image is removed from the filesystem using unlink from Node.js's module.

**Client-Side States:**

The **Menu Items State** not only stores fetched menu items, including all items or filtered results based on search criteria,… but it also ensures data consistency by verifying that the menu item name does not already exist in the database before new items are added or existing items are updated. This check helps avoid duplicate entries, maintaining the integrity of the menu data.

Filter/Search State: Holds search keywords or filter conditions (like name, type, price, or status) used to retrieve specific menu items.

Item Edit/Create State: Manages form data for adding or editing a menu item, including name, type, description, price, and the image.

API Status State: Tracks the status of API calls (loading, success, or error) to provide feedback to the user.

2. **Feature / Application page 2: Table**

**Description:**

– In staff role, the table management help staff organize and manage table. It provides table number and its seating capacity. Staff can quickly add new tables or remove existing ones.

– In customer role: The table management feature provides customer with tools to view the restaurant's table. This includes viewing tables.

**ScreenShots:**

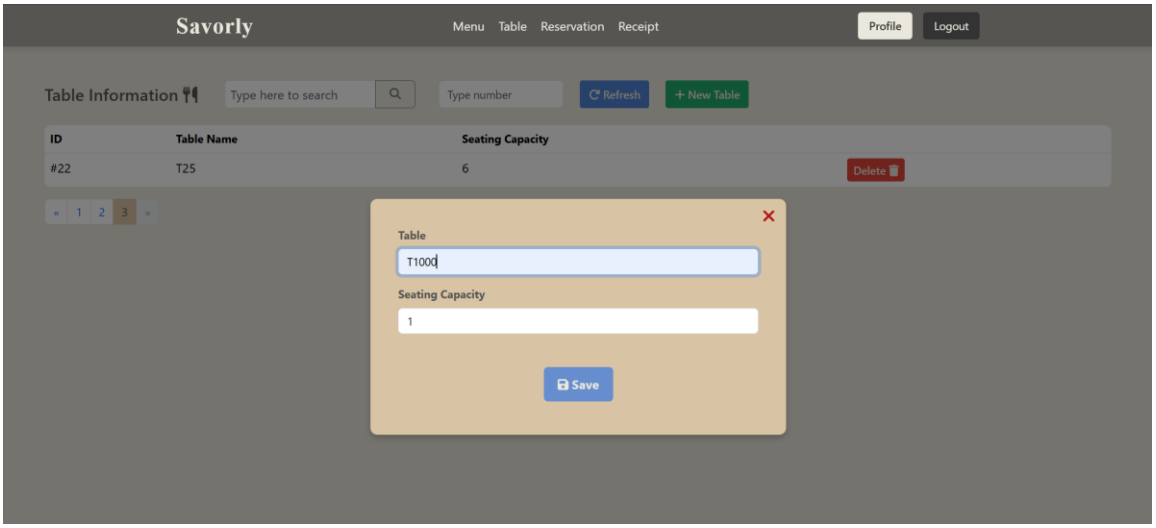


*Image 4. Table Form*

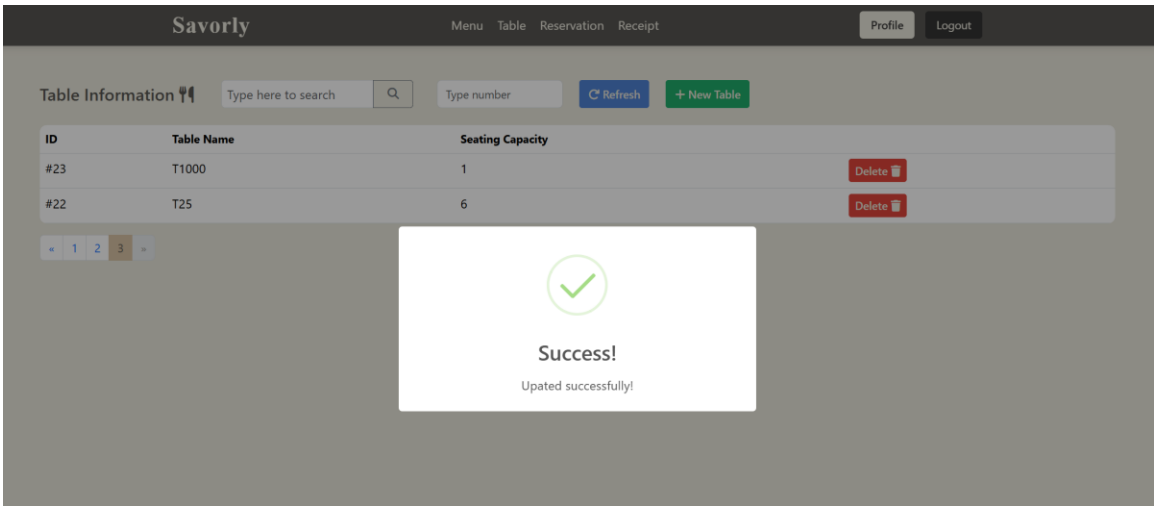


*Image 5. Table added successfully*

**Implementation details:**

<u>Libraries Used:</u>

- **Express**: A lightweight web framework used for creating API routes. It's responsible for handling HTTP requests and responses for the CRUD operations.
- **Knex**: A SQL query builder that simplifies database interactions, allowing me to perform SQL operations.
- **ApiError**: To handle errors.
- **fontawesome:** Provides icons for actions like adding, editing, and deleting tables, enhancing the table management UI. For instance, a trash icon for delete.
- **tanstack/vue-query:** Manages server-state for tables by fetching and caching data (the list of restaurant tables) and syncing changes (after adding or deleting a table). This ensures the data is always up-to-date without excessive API requests.
- **vee-validate/zod:** Validates forms used for adding or editing tables, ensuring input fields like table number and seating capacity meet the required criteria.
- **Bootstrap:** Styles the table management interface, including the layout of forms, the table list, and buttons.

- **SweetAlert2:** Displays user-friendly alerts and confirmation dialogs for actions like deleting a table or notifying staff when a table is successfully added or updated.
- **Vue:** The main framework used to create reactive and reusable components for managing tables (TableList, TableForm, TableCard,…).
- **Vue-Router:** Manages navigation within the table management interface, such as transitioning between a table components and table views

**Server-Side APIs:**

## POST /api/table/ByStaff

**Description:** Staff create a new table.

**Endpoint**: /api/table/ByStaff

**Form format**: multipart/form-data

**Request Body (required)** : table_number (string), seating_capacity (integer).

**Format Received:**


```
{
  "status": "success",
  "data": {
    "table": {
      "table_id": 0,
      "table_number": "string",
      "seating_capacity": 0
    }
  }
}
```


**Data storage**

Table information is stored in a tables table with fields like table_id, table_number, seating_capacity.

**Client-side states**

Allows staff to add table.

3. **Feature / Application page 3: Reservation**

**Description:**

- In staff role, reservation management simplifies booking and updating reservations. Staff can create new reservations by inputting customer details, selecting available tables, and scheduling the desired time. Status updates, such as confirmed, completed, or canceled, can be made seamlessly. Transactions ensure that all related records are accurately updated, maintaining data integrity.
- In customer role, reservation management simplifies booking and updating reservations. Customer can create new reservations by inputting customer details, selecting available tables, and scheduling the desired time. The system will automatically check and notify customers if the table has been booked, ensuring data consistency. Status updates, such as confirmed, completed, or canceled, can be made seamlessly. Transactions ensure that all related records are accurately updated, maintaining data integrity.

**ScreenShots:**

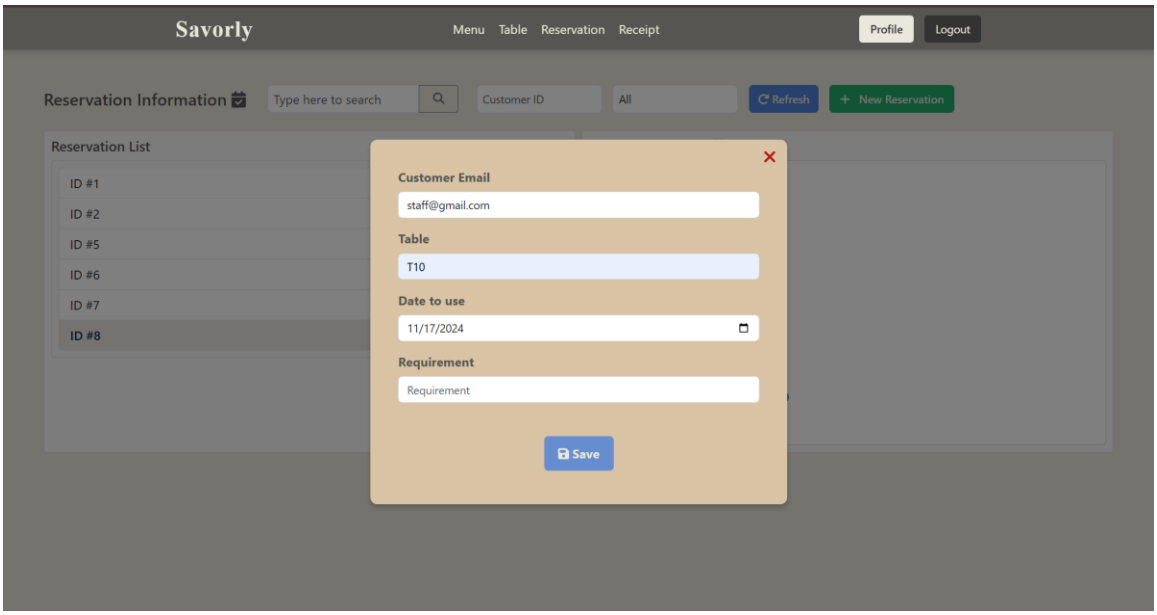


*Image 6. Staff create new reservation*

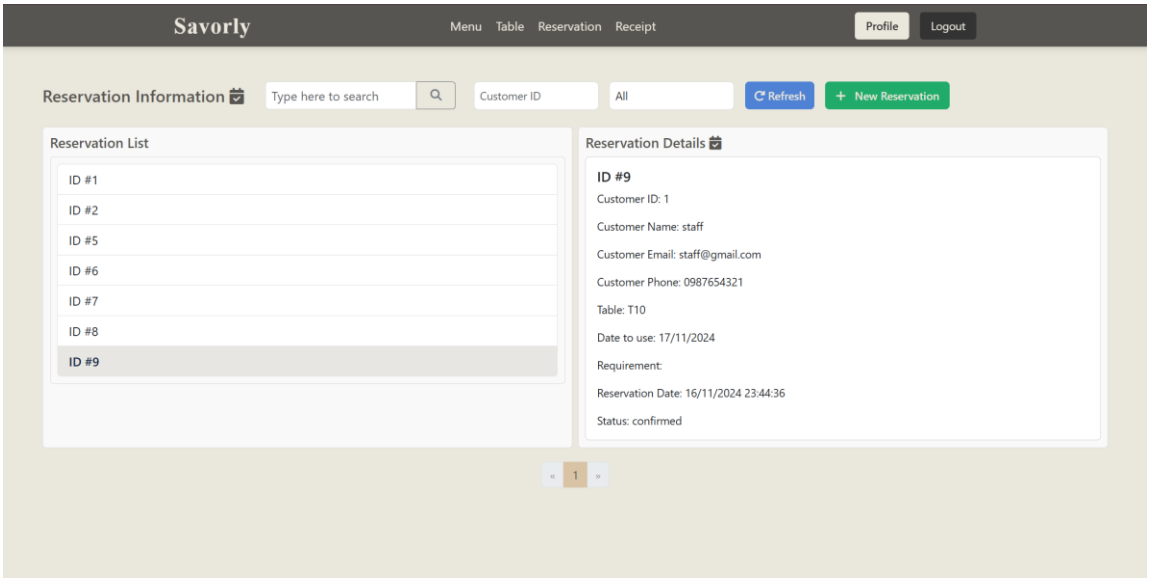


*Image 7. Result after staff creating new reservation*

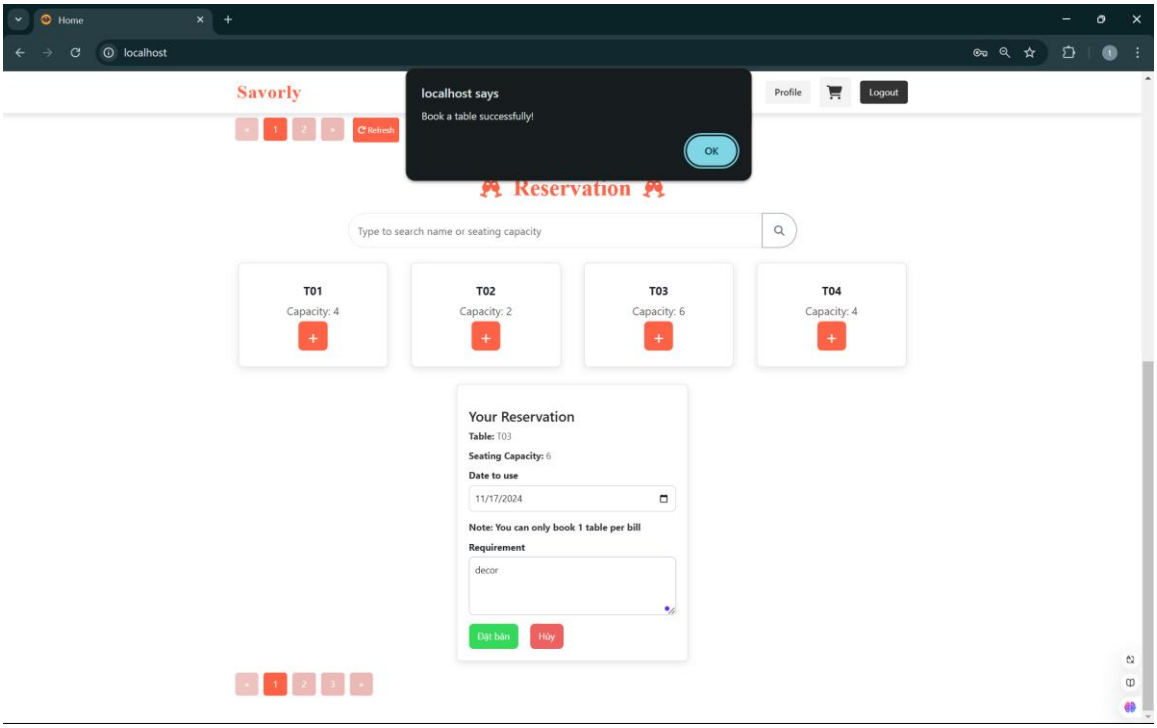


*Image 8. Customer creating new reservation successfully*

**Implementation details:**

<u>Libraries Used:</u>

- **Express**: A lightweight web framework used for creating API routes. It's responsible for handling HTTP requests and responses for the CRUD operations.
- **Knex**: A SQL query builder that simplifies database interactions, allowing me to perform SQL operations.
- **ApiError**: To handle errors.
- **fontawesome:** Provides icons for actions like creating, editing, or updating reservations, improving the visual design.
- **tanstack/vue-query:** Manages server-state for reservations by fetching reservation data and syncing changes (when reservation status is updated or a new reservation is created). This ensures real-time updates across the UI.
- **vee-validate/zod:** Validates reservation forms, ensuring fields like the reservation date, table selection, and special requests are correctly filled out before submission.
- **Bootstrap:** Styles the reservation management UI, including the reservation form layout, reservation list, and buttons for actions.
- **SweetAlert2:** Displays user-friendly alerts for actions such as confirming reservation creation or handling errors when required fields are missing.
- **Vue:** The main framework used to build reactive components for reservation management, such as resrvation card, reservation lists, reservation form and modals for adding reservations.
- **Vue-Router:** Handles navigation within the reservation management interface, such as transitioning between a reservation components and reservation views.

**Server-Side APIs:**

## POST /api/reservation/ByStaff

**Description:** Staff create a new reservtaion for customer.

**Endpoint**: /api/reservation/ByStaff

**Form format**: multipart/form-data

**Request Body (requiered)**: useremail (string), table_number (string), reservation_date (string ($date)), speacial_request (string).

**Format Received:**


```
{
  "status": "success",
  "data": {
    "reservation_id": 0,
    "reservation_date": "2024-11-17",
    "party_size": 0,
    "special_request": "string",
    "status": "booked",
    "user": {
      "username": "string",
      "useremail": "string"
    },
    "table": {
      "table_number": "string",
      "seating_capacity": 0,
      "table_status": "string"
    }
  }
}
```


## POST /api/receipts/addTable/

**Description:** Customer create a new reservtaion.

**Endpoint**: /api/receipts/addTable/

**Form format**: multipart/form-data

**Request Body (requiered)**: table_number (string), reservation_date (string ($date)), speacial_request (string).

**Format Received**:


```
{
  "status": "success",
  "data": {
```

```
    "contact": {
      "order_id": 0,
      "userid": 0,
      "staff_id": 1,
      "reservation_id": 0,
      "order_date": "2024-11-17",
      "total_price": 0,
      "status": "Pending"
    }
  }
}
```


**Data storage**

Reservation details is stored in a reservations table, possibly containing fields like reservation_id, user_email, reservation_date, and status.

**Client-side states**

The current reservation state for the user (active reservations, canceled reservations) and user authentication state to retrieve their reservations. For staff, they can also update status of the reservation automatically from the receipt status. The reservation state also handles reservation details such as customer names, times, and assigned tables, ensuring smooth synchronization with the server when changes occur.

4. **Feature / Application page 4: Receipt**

**Description:**

- In staff role, receipt management allows staff to track the receipt efficiently. Receipts can be created directly from reservations. Staff can update receipt statuses to reflect payment completion or cancelation. Transactions ensure consistency across linked data, such as reservations and tables.
- In customer role, receipt management allows customers can track the receipt efficiently. Customers can create new reservation by providing order dates and optional reservation, and view their invoices with filters. For cart management, they can add items to their cartafter verifying item availability or remove items if the items are present in the cart. Regarding reservations, customers can book tables by providing table numbers and reservation dates, with the system ensuring table

availability and preventing duplicate bookings. They can confirm orders and reservations to finalize pending orders or cancel existing bookings or orders. Additionally, customers can view the details of their pending orders, including the receipt, reservation, items, and table details. Overall, these features enable customers to seamlessly manage their purchasing experience, from creating and managing orders to handling reservations, with all actions validated for accuracy and reliability.

**ScreenShots:**

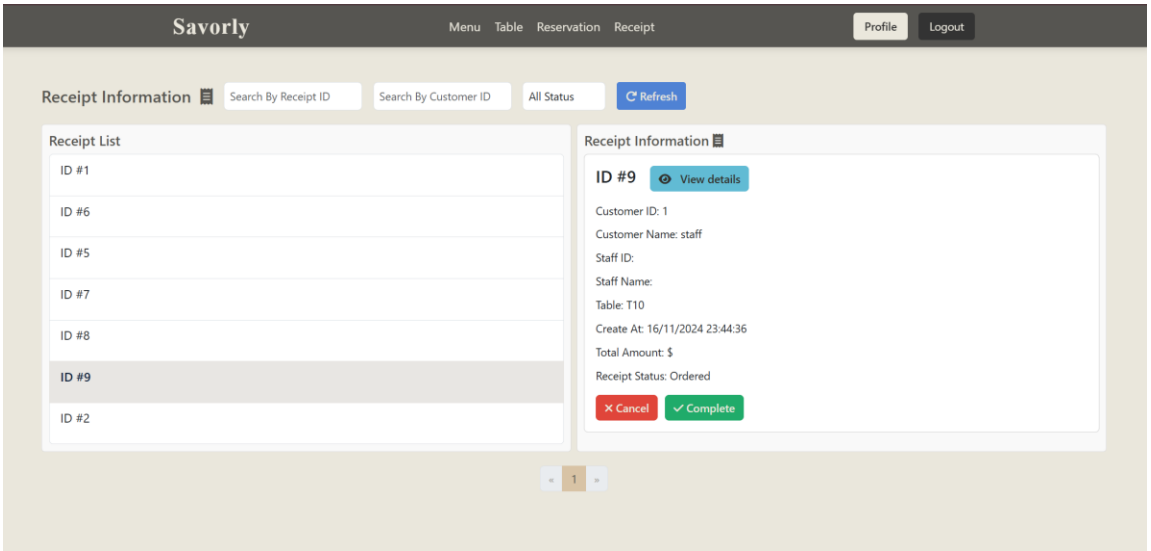


*Image 9. Receipt before staff updating status*

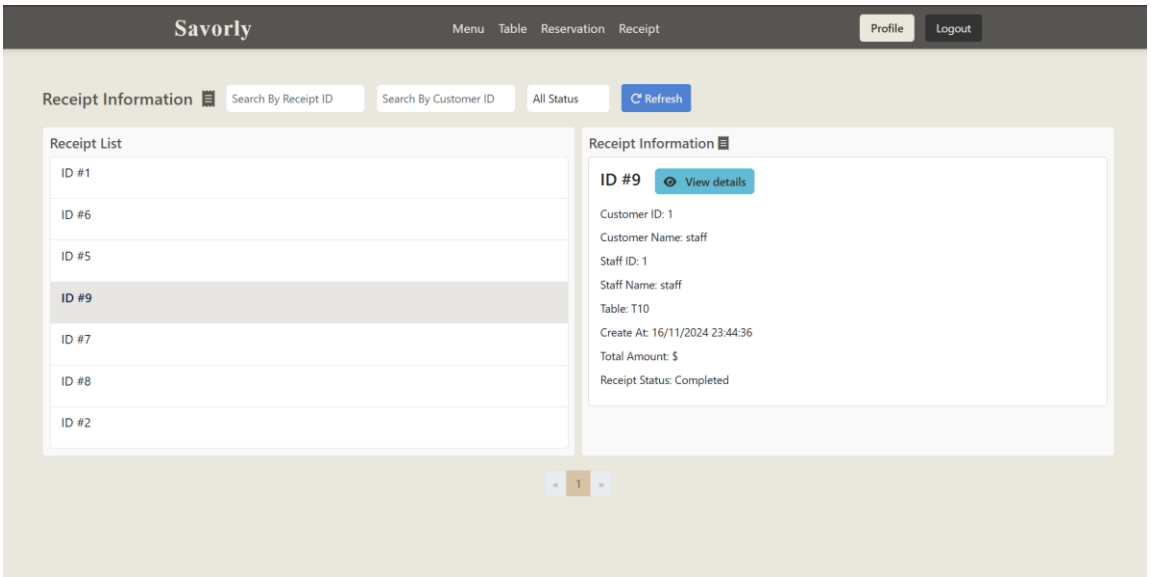


*Image 10. Receipt after staff updating to complete*

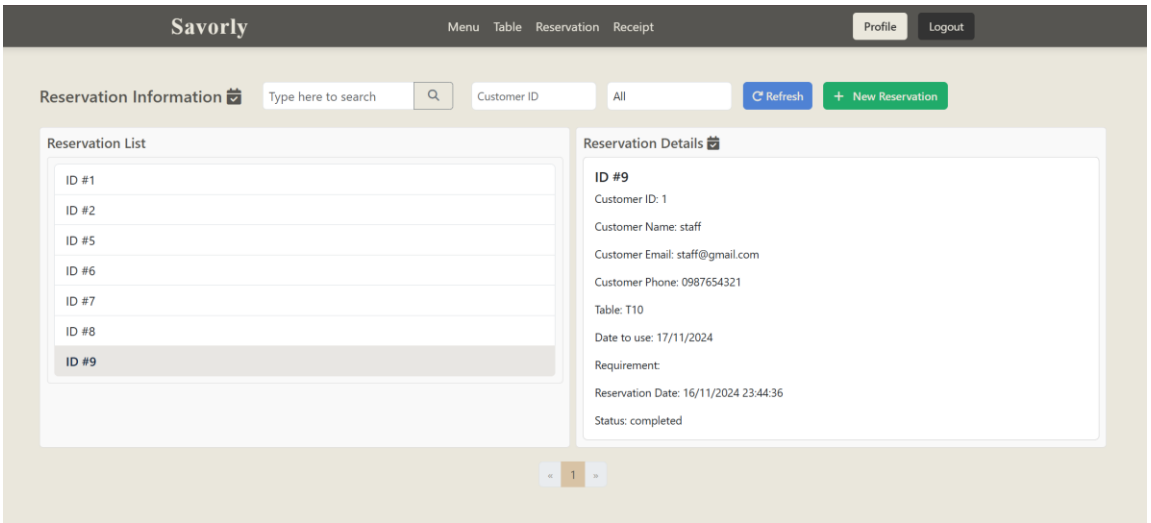


*Image 11. Reservation after staff updating receipt status*

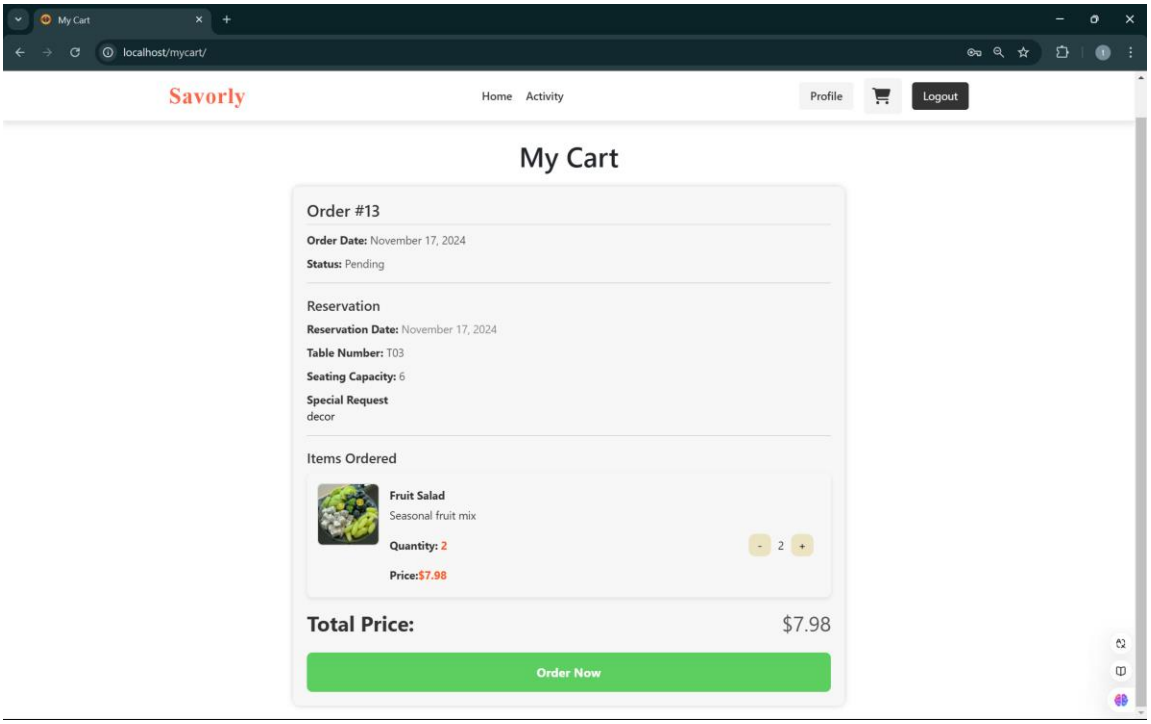


*Image 12. Reservation and Ordering Food before customer updating receipt*

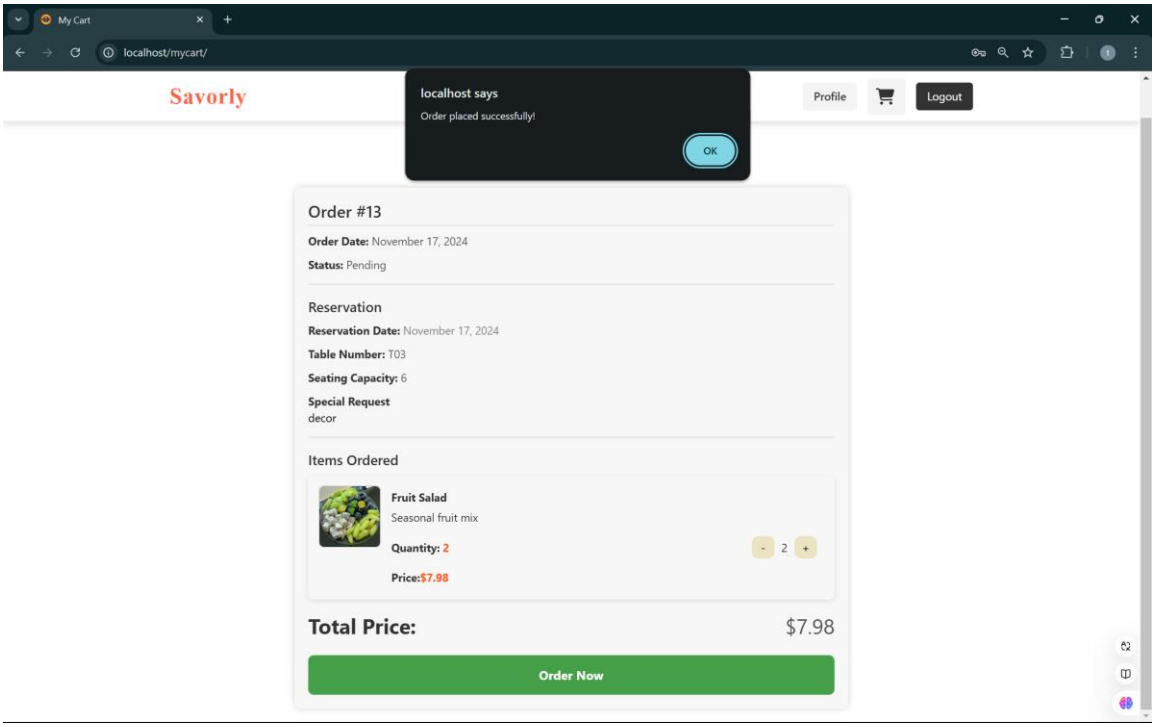


*Image 13. Reservation and Ordering Food after customer updating receipt status*

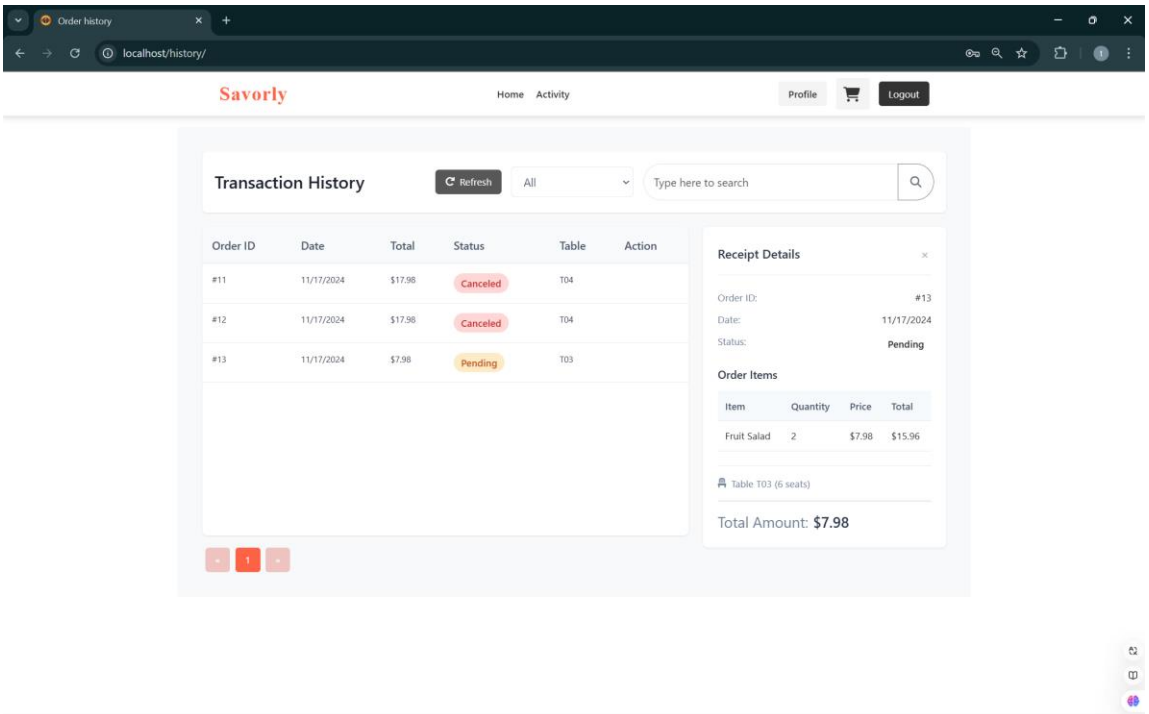


*Image 14. Reservation and Ordering Food before customer updating receipt status in History*

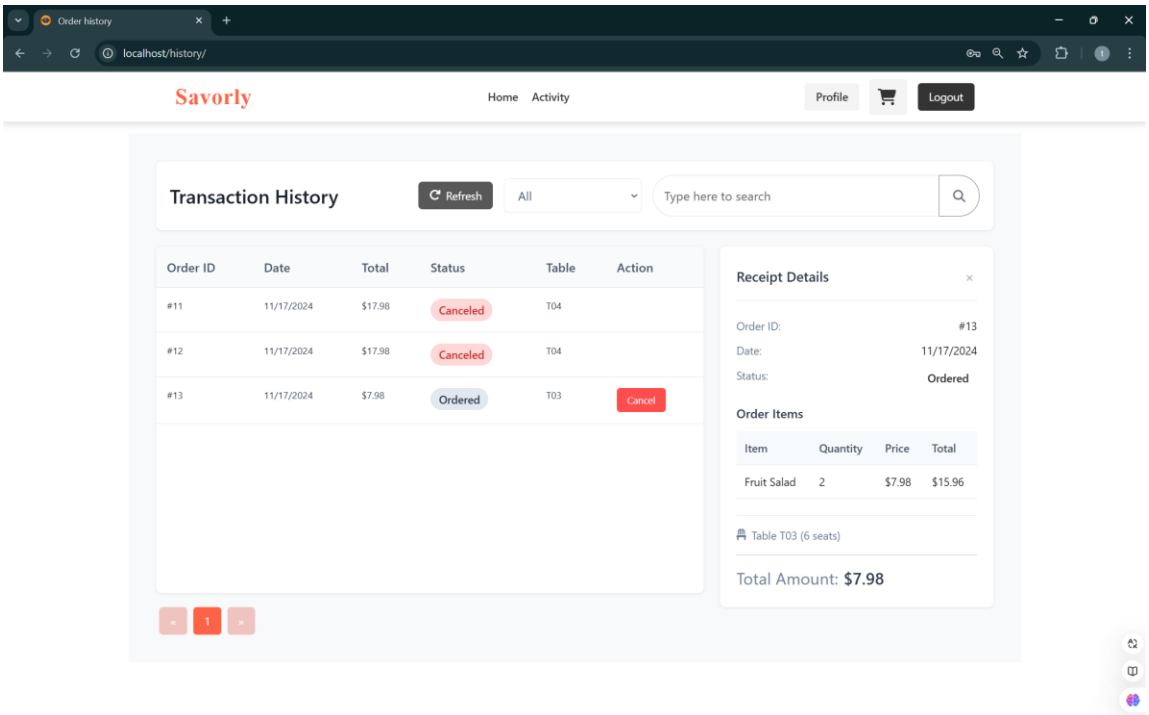


*Image 15. Reservation and Ordering Food after customer updating receipt status in History*

**Implementation details:**

<u>Libraries Used:</u>

- **Express**: A lightweight web framework used for creating API routes. It's responsible for handling HTTP requests and responses for the CRUD operations.
- **Knex**: A SQL query builder that simplifies database interactions, allowing me to perform SQL operations.
- **ApiError**: To handle errors.
- **fontawesome**: Provides icons for actions such as creating a receipt, editing receipt status, or viewing details.
- **tanstack/vue-query**: Manages server-state for receipts by fetching receipt data and syncing changes (when the receipt status is updated). This ensures real-time updates and reduces redundant API calls.

- **Bootstrap**: Styles the receipt management interface, including form layouts for lists of receipts, details of receipts, and buttons for user actions, ensuring a responsive and visually appealing design.
- **SweetAlert2**: Displays confirmation dialogs and alerts for actions like confirming, editing receipt status, or handling errors when required fields are missing.
- **Vue**: The main framework used to create reactive components for receipt management, such as receipt cards, receipt lists, and modals for viewing receipt details.
- **Vue-Router**: Handles navigation within the receipt management interface, allowing smooth transitions between receipt components and receipt views.

**Server-Side APIs:**

## PUT / api/receipts/verify/ByStaff/{order_id}

**Description:** Change status of a specific receipt.

**Endpoint**: /api/receipts/verify/ByStaff/{order_id}

**Form format**: multipart/form-data

**Parameters (required):** order_id (integer)

**Request Body (required)**: status (string)

**Format Received:**


```
{
  "status": "success",
  "data": {
    "receipt_info": {
      "order_id": 0,
      "userid": 0,
      "user_name": "string",
      "staff_id": 0,
      "staff_name": "string",
      "reservation_id": 0,
      "table_id": "string",
      "order_date": "2024-11-17T05:16:18.451Z",
```

```
      "total_price": 0.1,
      "status": "Pending",
      "order_items": [
        {
          "order_item_id": 0,
          "item_id": 0,
          "item_name": "string",
          "quantity": 0,
          "item_price": 0,
          "price": 0.1
        }
      ]
    }
  }
}
```


## POST /api/receipts/addItem

**Description:** Adding item to Cart

**Endpoint**: /api/receipts/addItem

**Form format**: multipart/form-data

**Request Body (required)**: item_id (integer), quantity (number)

**Format Received:**


```
{
  "status": "success",
  "data": {
    "customer": {
      "order_item_id": 0,
      "order_id": 0,
      "item_id": 0,
      "quantity": 0,
      "price": 0.1
    }
  }
```

}

## PUT /api/receipts/customer/verify/

**Description:** Customer verify ordering

**Endpoint**: /api/receipts/customer/verify/

**Form format**: multipart/form-data

**Request Body (required)**: status (string)

**Format Received:**


```
{
  "status": "success",
  "data": null
}
```


**Data storage**

Receipt details are stored in a receipt table, possibly containing fields like order_id, userid, staff_id, reservation_id, order_date, total_price, and status.

**Client-side states**

The system manages receipts and items for a restaurant-style order process, where client-side state includes details such as user ID, staff ID, reservation ID, order date, total price, and status. When a user adds an item to their receipt, the system first checks if there is an existing "Pending" receipt for the user. If none exists, a new one is created. Items can then be added to the receipt, with their quantities and total prices updated accordingly. If an item is removed or its quantity is reduced to zero, the system adjusts the total price, and if no items remain, the receipt is deleted. Additionally, the system allows the status of an order to be updated (e.g., from "Pending" to "Ordered" or "Canceled") depending on the user's actions. All operations are handled within database transactions to ensure consistency, and users are provided with clear feedback messages after each action.

5. **Feature / Application page 5: User**

**Description:**

In customer role, user management allows manage user accounts securely. Customer can create new user by validating user information, including email

format, phone number, and password strength. Customers, after authenticating via the login feature can manage their profiles using several functionalities. They can view their profile information, update personal details, and securely logout to end their session. The system ensures only authenticated users access sensitive operations, protecting user data. Moreover, customers have the option to delete their accounts permanently if they wish to stop using the system. Comprehensive input validation and session management enhance security and user experience.

**Screenshots:**

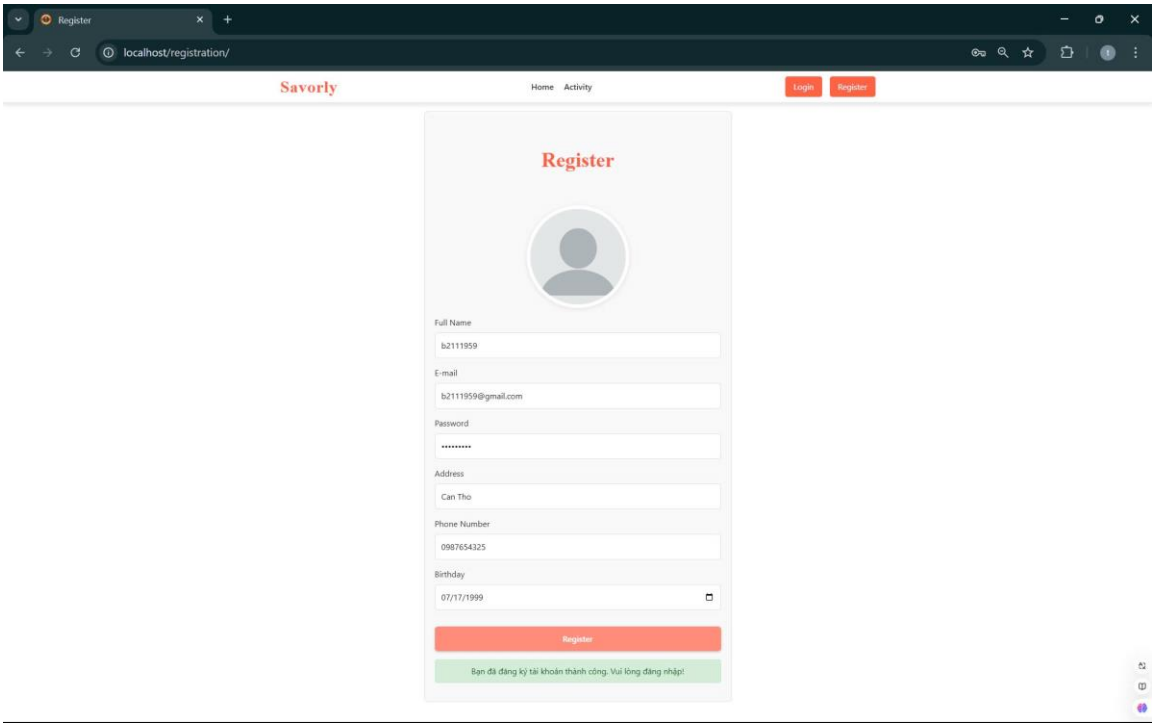


*Image 16. Register successfully*

**Implementation details:**

<u>Libraries Used:</u>

- **Express**: A lightweight web framework used for creating API routes. It's responsible for handling HTTP requests and responses for the CRUD operations.
- **Knex**: A SQL query builder that simplifies database interactions, allowing me to perform SQL operations.
- **ApiError**: To handle errors.

- **fontawesome**: Provides icons for actions such as creating a receipt, editing receipt status, or viewing details.
- **tanstack/vue-query**: Manages server-state for receipts by fetching receipt data and syncing changes (when the receipt status is updated). This ensures real-time updates and reduces redundant API calls.
- **Bootstrap**: Styles the receipt management interface, including form layouts for lists of receipts, details of receipts, and buttons for user actions, ensuring a responsive and visually appealing design.
- **vee-validate/zod:** Validates reservation forms, ensuring fields like the reservation date, table selection, and special requests are correctly filled out before submission.
- **Vue**: The main framework used to create reactive components for receipt management, such as receipt cards, receipt lists, and modals for viewing receipt details.
- **Vue-Router**: Handles navigation within the receipt management interface, allowing smooth transitions between receipt components and receipt views.

**Server-Side APIs:**

## POST /api/users/registration/

**Description:** Customer create accounts

**Endpoint**: /api/users/registration/

**Form format**: multipart/form-data

**Request Body (required)**: userrole (integer) – default =1 and does not allow user input, username (string), userbirthday (string $date), userphone (string), useremail (string $email), userpwd (string $password), useraddress (string) and useravatarFile (string $binary)

**Format Received:**


```
{
  "status": "success",
  "data": {
    "contact": {
```

"userid": 0,
"userrole": 1,
"username": "string",
"userbirthday": "2024-11-17",
"userphone": "string",
"useremail": "user@example.com",
"userpwd": "********",
"useraddress": "string",
"useravatar": "string"
}
}
}

**Data storage**

User details is stored in a users table, including fields like email, and other personal information.

**Client-side states**

This feature reads and stores data in the users table. The users fields include userid, userrole, username, userbirthday, userphone, useremail, useraddress,... All database operations are managed using Knex. Image files are uploaded via Multer and stored on the server. When a user is deleted, the associated image is removed from the filesystem using unlink from Node.js's module.